Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #11

CS@TUCN

TECHNICAL UNIVERSITY OF CLUJ-NAPOCA

Agenda

- **Constructs and relationship with graphs**
  - For all is true
  - Application – same graph
- **Search for a path in graphs**
  - dfs
  - bfs (v1 – other versions later)
- **Graphs isomorphism**
  - with metaprogramming
  - with nonmonotonic reasoning
  - various implementations (compare & contrast analysis)

[q] ?- for_all_is_true(member(Char, ['a','b','c']), char_type(Char, lower)).
[q] ?- for_all_is_true(member(Char, ['a','B','c']), char_type(Char, lower)).

For all is true technique

- Construct to check if `for_all Predicate1 is_true Predicate2`
- Aims to identify/verify whether in all cases when P1 is true, P2 is true as well. Thus, checks P1=>P2.
- Template looks like:

```
for_all_is_true(X,Y):-
      X,                    % calls X, a predicate,
                            %       whose execution may instantiate hidden args
                            % calls Y in the same context. If succeeds, its negation fails,
      not(Y),!,             %       and backtracks to X who is executed again, with a
                            %       new instance of args; generates a new context
                            %       the first context of X where Y fails, succeeds, and !
                            %       is irreversible crossed and the predicate fails
      fail.                 %       overall.

                            % if we reached here, in all contexts when X holds true
for_all_is_true(_,_).       % succeeds as well, therefore, X=>Y
```

For all is true in graphs context

- Given graphs G1 and G2 with neighbor and edge representations respectively, do they represent the same graph?

G1

```
neighbor(Node,List_of_Neighbors)
```

G2

```
edge(Node1,Node2)
```

- G1 and G2 same means G1⇔G2 which is (G1=>G2)+(G2=>G1)

(G1=>G2)

```
for_all edges_in_G1
is_true edge_in_G2
```

(G2=>G1)

```
for_all edges_in_G2
is_true edge_in_G1
```

- Define the edges in the 2 representations:

An edge in G1: (defines the nondeterministic check for all edges)

```
is_edge_1(X,Y):-
        neighbor(X,L),
        member(Y,L).
```

% to find all edges in G1, call nondeterminist(X,Y,free)

% finds first pair first (and next on backtrack), instantiates both X and L.

% nondeterm call on member; instantiate Y, one at a time, eventually all.

An edge in G2:

```
is_edge_2(X,Y):-
        edge(X,Y);
        edge(Y,X).
```

Same graph?

- (G1=>G2)

```
for_all edges_in_G1
is_true edge_in_G2
```

(G2=>G1)

```
for_all edges_in_G2
is_true edge_in_G1
```

- Denote the `for_all_is_true` as `eq1` and `eq2` depending on the implication (G1=>G2) respectively (G2=>G1) we verify

```
eq1:-
        is_edge_1(X,Y),          % for each edge in the first representation
        not(is_edge_2(X,Y)),!,   % there is one edge in the other graph
        fail.                    % otherwise, we reach here, hence, fail.
eq1.                             % if we get here, G1=>G2 shown

eq2:-
        is_edge_2(X,Y),
        not(is_edge_1(X,Y)),!,
        fail.
eq2.

same_graph:- eq1, eq2.
```

```
edge(a,b).
edge(b,a).
neighbor(a,[b]).
neighbor(b,[a]).

[q] ?- same_graph.
```

[q] ?- gen_graph2, listing(neighbor/2).

Change representation of a graph

- Given graph **G1** a weighted, edge representation and we need to get **G2** with neighbor representation

G1

```
edge(a,b,15).
```

G2

```
neighbor(a,[p(b,15),…]).
```

```
gen_graph2:-
        is_edge(X,_,_),
        not(neighbor(X,L)),
        findall(Z,succ(X,Z),L),
        assertz(neighbor(X,L)),
        fail.
gen_graph2.

succ(X,Z):-
        is_edge(X,Y,W),
        Z=p(Y,W).
```

- Note:

`findall(Z,succ(X,Z),L)` same as `findall(p(Y,W), is_edge(X,Y,W),L).`

- Given G2, generate G1. Homework.

Depth first search

- Assume undirected, unweighted graph, edge representation
- 2 stage process:
  1. 1st clause: make the trail placing in a queue the vertices in order of visitation (being in the additional knowledge base `vert` - hence a dynamic predicate - is as if we have a grey vertex).
  2. 2nd clause: when done, collect them.

```
df_search(X,_):-
        assertz(vert(X)),       % place start/current node in Q
        is_edge(X,Y),           % take first/next neighbor of current node X
        not(vert(Y)),           % if already visited, backtrack to take another;
        df_search(Y,_).         %          if not, continue from Y
df_search(_,L):-
        assertz(vert(end)),
        collect([],L).          % similar to collect in findall, but on vert akb.
```

- It is covering just the connected component of the starting vertex (is similar to dfs_visit in Cormen).
  - Need a wrapper to re-run it from all connected components (all remaining white nodes – color NOT implemented here).

Breadth first search

- Assume undirected, unweighted graph, edge representation
- Queue made by the akb named `node` (hence dynamic predicate)

```
bf_search(X,_):-
        assertz(node(X)),       % add at the end of the Q the current node
        node(Y),                % reads from front of Q, gets Y instantiated (initially X=ONLY one in Q)
        is_edge(Y,Z),           % take first/next neighbor of current Y (gets Z instantiated);
                                %           if none, backtrack and take next from Q
        not(node(Z)),           % if Z already visited, backtrack to take another;
        assertz(node(Z)),       % if not, put Z in Q and
        fail.                   % backtrack anyway to another neighbor of Y
bf_search(_,L):-
        assertz(node(end)),
        collect([],L).          % similar to collect in findall, but on node akb.
```

- There is an issue with this implementation for some languages/dialects (due to an optimization strategy). Oral explanation. Fix it! Homework.

Graphs isomorphism problem statement

- Graphs are essentially "the same" in structure, differing only by the names or arrangement of their vertices.
- Let us:
  - G1=(V1,E1)
  - G2=(V2,E2)
  - G1 iso G2 iff
    i) |V1|=|V2|
    ii) $\exists\ \varphi$: V1-> V2 a bijection so that
    iii) $\forall$ x, y $\in$V1 [(x, y) $\in$ E1 $\Leftrightarrow$ (φ(x), φ(y)) $\in$ E2)
  - In simpler words: φ matches up vertices one-to-one while preserving adjacency

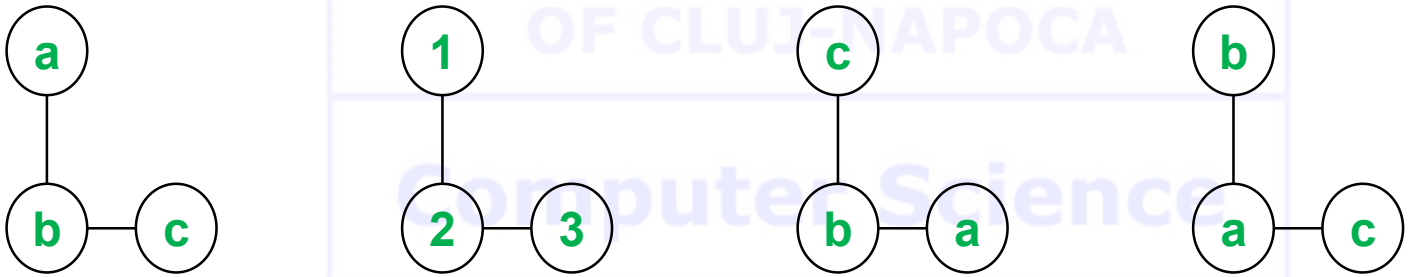
- Ex: Are the 2 graphs isomorphs?

```
graph1([
        n(a,[b,c,d]),
        n(b,[a,c]),
        n(c,[a,b,d]),
        n(d,[a,c])
]).
```

```
graph2([
        n(1,[2,4]),
        n(2,[1,3,4]),
        n(3,[2,4]),
        n(4,[1,2,3])
]).
```

Graphs isomorphism
Simple example

- Consider these two graphs:
  - **Graph A** has vertices labelled {a,b,c} with edges (a,b),(b,c).
  - **Graph B** has vertices labelled {1,2,3} with edges (1,2),(2,3).



Graphs isomorphism approach (v1)

```
?-graph1(G1),graph2(G2),iso_graph1(G1,G2).

iso_graph1(L1,L2):-eq_perm(L1,L2,eq_neighb).

eq_perm([H1|T1],L2,EQ):-      % is the perm predicate with an equivalence
       delete(H2,L2,T2),      % nondeterministic behavior. Implemented how?
       P=..[EQ,H1,H2],        % which is created here (metapredicate)
       call(P),               % and called here to execute
       eq_perm(T1,T2,EQ).
eq_perm([],[],_).             % succeeds iff sets have the same cardinality
% so an immediate improvement would check cardinalities beforehand

eq_neighb(n(N1,L1),n(N2,L2)):-   % 2 neighb pairs are equivalent
       eq_node(N1,N2),           % if the nodes are equivalent
       eq_perm(L1,L2,eq_node).   % and the neighb lists are equivalent

delete(H,[H|T],T).               % what if we add a cut here?
delete(X,[H|T],[H|R]):-
       delete(X,T,R).            % what if we add the [] clause?
```

Nodes equivalence
(implements φ function)
V1 – default logic with side effects

- p a dynamic predicate, used for the nomonotonic reasoning
- implements φ function (to form `p` as akb)

```
eq_node(N1,N2):-    % if nodes N1 and N2 already form a pair in the akb
      p(N1,N2).     % the evaluation continues
eq_node(N1,_):-     % if N1 forms a pair with some OTHER node
      p(N1,_),!,    % we get an inconsistency, so should NOT allow
      fail.         % (N1,N2) form a pair, so, fail to backtrack
eq_node(_,N2):-     % symmetric on N2
      p(_,N2),!,    % not an injective function
      fail.         % How/where bijection is insured?
eq_node(N1,N2):-    % if you reach here, the is no inconsistency
      asserta(p(N1,N2)).          % hence pair N1, N2 is legitimate.
eq_node(N1,N2):-                  % if at a later moment,
      retract(p(N1,N2)),!,        % although pair N1, N2 is legitimate,
                                  % the reasoning cannot conclude,
                                  % so remove it from the akb, and
      fail.                       % fail to backtrack and resume execution
                                  % WITHOUT the pair in the akb
```

Graphs isomorphism approach (v1)

```
?- graph1(G1),graph2(G2), iso_graph1(G1,G2).

iso_graph1(L1,L2):-eq_perm(L1,L2,eq_neighb).
eq_perm([H1|T1],L2,EQ):-
                delete(H2,L2,T2),
                P=..[EQ,H1,H2],
                call(P),
                eq_perm(T1,T2,EQ).
eq_perm([],[],_).

eq_neighb(n(N1,L1),n(N2,L2)):-
                eq_node(N1,N2),
                eq_perm(L1,L2,eq_node).

delete(H,[H|T],T).
delete(X,[H|T],[H|R]):-delete(X,T,R).

eq_node(N1,N2):-p(N1,N2).
eq_node(N1,_):-p(N1,_),!,fail.
eq_node(_,N2):-p(_,N2),!,fail.
eq_node(N1,N2):-asserta(p(N1,N2)).
eq_node(N1,N2):-retract(p(N1,N2)),!,fail.
```

Nodes equivalence
(implements φ function)
V2 – with arguments (lists)

```
?- graph1(G1),graph2(G2), iso_graph2(G1,G2).
% instead of the akb p in v1, here we store the pairs in the arg list
%  arg 4 = partial list, empty initially (arg to model the akb)
% arg 5 = final list; a copy of the partial list at the end of the execution
%                  (arg with the status of the akb at the end; the bijection)
iso_graph2(L1,L2):-eq_perm(L1,L2,eq_neighb,[],Lout).

eq_perm([H1|T1],L2,EQ,LI,LO):-      % same as in v1
       delete(H2,L2,T2),
       P=..[EQ,H1,H2,LI,Lint],      % just that with args
       call(P),
       eq_perm(T1,T2,EQ,Lint,LO).
eq_perm([],[],_,L,L).

eq_neighb(n(N1,L1),n(N2,L2),LI,LO):-
       eq_node(N1,N2,LI,Lint),
       eq_perm(L1,L2,eq_node,Lint,LO).
```

Nodes equivalence
(implements φ function)
V2 – with arguments (lists) – contd.

- p is a pair of nodes in the list argument
- list evolves nomonotonic (during reasoning) to allow for pairs addition/removal
- models φ function

```
eq_node(N1,N2,LI,LI):-
        member(p(N1,N2),LI), !.
eq_node(N1,N2,LI,[p(N1,N2)|LI]):-
        not(member(p(N1,_),LI)),
        not(member(p(_,N2),LI)).
```

- Those 2 clauses do the same as those 5 in v1

Nodes equivalence
(φ function v2 VS v1)

```
eq_node(N1,N2,LI,LI):-
        member(p(N1,N2),LI),!.

eq_node(N1,N2,LI,[p(N1,N2)|LI]):-

        not(member(p(N1,_),LI)),

        not(member(p(_,N2),LI)).
```

```
eq_node(N1,N2):-
        p(N1,N2).
eq_node(N1,_):-
        p(N1,_),!,
        fail.
eq_node(_,N2):-
        p(_,N2),!,
        fail.
eq_node(N1,N2):-
        asserta(p(N1,N2)).
eq_node(N1,N2):-
        retract(p(N1,N2)),!,
        fail.
```

Nodes equivalence
(implements φ function)
V3 – with arguments (incomplete lists)

- Same as v2 just that lists are incomplete
- So, it must adjust the behavior of the search predicate to handle appropriately the "not found" case
- Just the equivalence changes (and the `eq_perm` predicate needs just 1 list arg):

```
eq_node(N1,N2,[p(N1,N2)|_]):-!.
eq_node(N1,_,[p(N1,_)|_]):- !,
      fail.
eq_node(_,N2,[p(_,N2)|_]):- !,
      fail.
eq_node(N1,N2,[_|T]):-
      eq_node(N1,N2,T).
```

- Compare v3 with v1. Why is 1 (from v1) clause missing (in v3)?
- Compare v3 with v2.

Nodes equivalence
(φ function v3 VS v1)

```
eq_node(N1,N2,[p(N1,N2)|_]):-!.
eq_node(N1,_,[p(N1,_)|_]):-!,
        fail.
eq_node(_,N2,[p(_,N2)|_]):-!,
        fail.
eq_node(N1,N2,[_|T]):-

        eq_node(N1,N2,T).
```

```
eq_node(N1,N2):-
        p(N1,N2).
eq_node(N1,_):-
        p(N1,_),!,
        fail.
eq_node(_,N2):-
        p(_,N2),!,
        fail.
eq_node(N1,N2):-
        asserta(p(N1,N2)).
eq_node(N1,N2):-
        retract(p(N1,N2)),!,
        fail.
```

Nodes equivalence
(φ function v3 VS v2)

```
eq_node(N1,N2,[p(N1,N2)|_]):-!.
eq_node(N1,_,[p(N1,_)|_]):-!,
        fail.
eq_node(_,N2,[p(_,N2)|_]):-!,
        fail.
eq_node(N1,N2,[_|T]):-

        eq_node(N1,N2,T).
```

```
eq_node(N1,N2,LI,LI):-
        member(p(N1,N2),LI),!.

eq_node(N1,N2,LI,[p(N1,N2)|LI]):-

        not(member(p(N1,_),LI)),

        not(member(p(_,N2),LI)).
```

Nodes equivalence
(implements φ function)
V4 – with arguments (incomplete lists)

- Same as v2 just that lists are incomplete
- So, it must adjust the behavior of the search predicate to handle appropriately the "not found" case
- Just the equivalence changes (and the `eq_perm` predicate needs just 1 list arg):

```
eq_node4(N1,N2,LI):-
       member_il(p(N1,N2),LI),!.
eq_node4(N1,N2,[p(N1,N2)|LI]):-
       not(member_il(p(N1,_),LI)),
       not(member_il(p(_,N2),LI)).

member_il(_, L):-var(L), !, fail.
member_il(X, [X|_]):-!.
member_il(X, [_|T]):-member_il(X, T).
```

- Compare v4 with v2.

Nodes equivalence
(φ function v4 VS v2)

```
eq_node(N1,N2,LI):-
        member_il(p(N1,N2),LI),!.

eq_node(N1,N2,[p(N1,N2)|LI]):-

        not(member_il(p(N1,_),LI)),

        not(member_il(p(_,N2),LI)).
```

```
eq_node(N1,N2,LI,LI):-
        member(p(N1,N2),LI),!.

eq_node(N1,N2,LI,[p(N1,N2)|LI]):-

        not(member(p(N1,_),LI)),

        not(member(p(_,N2),LI)).
```